LogiStore

Optimisation des flux asynchrones et benchmark analytique haute performance (PostgreSQL vs DuckDB/Parquet).

Hedi Zarrouk
Yann Sasse
Tanjona R

CONTEXTE MÉTIER & PROBLÉMATIQUE

- **Casse système :** Le pipeline monolithique s'effondre lors des pics de charge.
- **Asynchronisme fatal :** Les mouvements reçus avant la création du produit font crasher le batch.
- **Pertes silencieuses :** Les échecs d'insertion sont ignorés, créant des écarts d'inventaire majeurs.
- **Zéro traçabilité :** Impossible de savoir où et pourquoi une donnée a été rejetée.

"Une donnée perdue, c'est un produit fantôme ou une rupture non anticipée."

CONTEXTE MÉTIER & PROBLÉMATIQUE

- **Casse système :** Le pipeline monolithique s'effondre lors des pics de charge.
- **Asynchronisme fatal :** Les mouvements reçus avant la création du produit font crasher le batch.
- **Pertes silencieuses :** Les échecs d'insertion sont ignorés, créant des écarts d'inventaire majeurs.
- **Zéro traçabilité :** Impossible de savoir où et pourquoi une donnée a été rejetée.

"Une donnée perdue, c'est un produit fantôme ou une rupture non anticipée."

La Solution Industrielle

Introduction d'une logique "Data Contract + Reject/Replay" pour transformer une fragilité technique en robustesse métier.

- Intégrité transactionnelle garantie
- Reprise automatique des anomalies

Principe de conception

Séparation

Découpler radicalement l'ingestion opérationnelle des requêtes analytiques métier.

Zéro Perte (DLQ)

Mise en place d'une logique de quarantaine stricte pour les données désynchronisées.

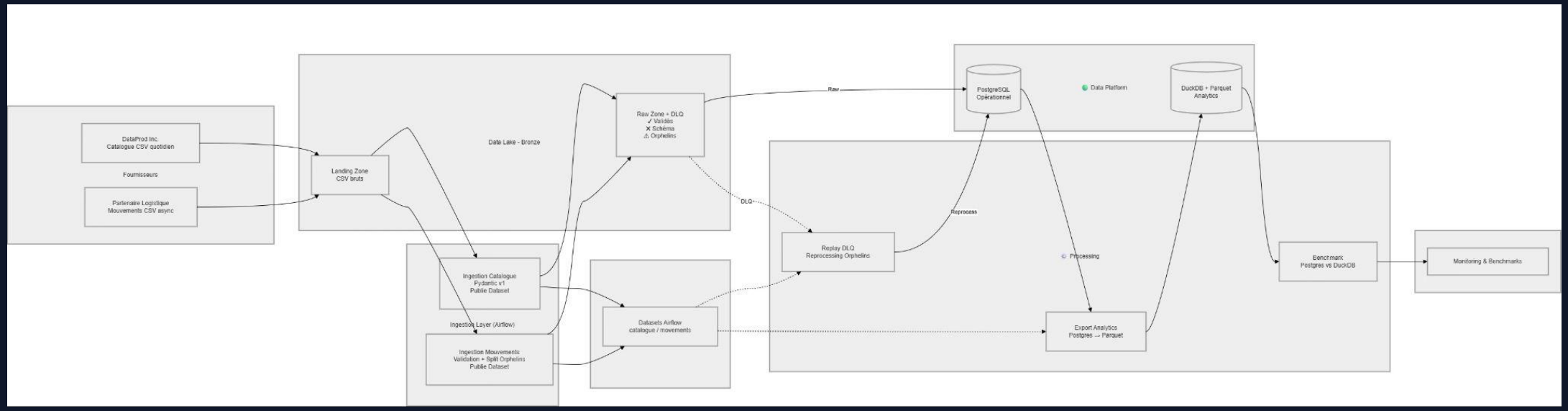
Agilité

Validation aux frontières du système et temps de réponse ultra-rapides pour l'analytique.

Architecture : La stack

- **Orchestration (Apache Airflow) :** Pilotage asynchrone, gestion du lignage et API Datasets.
- **Contrats (Pydantic) :** Validation stricte des schémas et typages JSON entrants.
- **OLTP (PostgreSQL) :** Base opérationnelle avec garanties ACID et intégrité relationnelle.
- **OLAP (DuckDB + Parquet) :** Format de stockage orienté colonnes pour des performances vectorisées.
- **Ops (Docker & CI/CD) :** Environnement reproductible couplé à GitHub Actions.

Architecture : La stack



Orchestration (Apache Airflow) : Pilotage asynchrone, gestion du lignage et API Datasets.

Contrats (Pydantic) : Validation stricte des schémas et typages JSON entrants.

OLTP (PostgreSQL) : Base opérationnelle avec garanties ACID et intégrité relationnelle.

OLAP (DuckDB + Parquet) : Format de stockage orienté colonnes pour des performances vectorisées.

Ops (Docker & CI/CD) : Environnement reproductible couplé à GitHub Actions.

GOUVERNANCE DES SCHÉMAS

Contrats Pydantic

Bloque le "Garbage In, Garbage Out"

Chaque ligne ingérée est validée contre un contrat versionné :

Typage strict : Validation automatique des flux JSON entrants avant toute interaction avec la base.

Règles métier : Vérification structurelle (ex: quantités toujours positives, formats ISO).

- ✓ Format SKU strict : SKU-XXXXX
- ✓ Enums contrôlés (Unité, Catégorie)
- ✓ Business Rules (Quantités négatives en sortie)

Versioning : Les contrats d'interface évoluent de manière sécurisée et documentée.

Bénéfice Clé

"La détection précoce des erreurs permet un rejet explicite avec motif, assurant la stabilité entre producteurs et consommateurs de données."

PASSAGE V1 / V2 DU CATALOGUE

Évolution du Schéma

Transition fluide de la version 1.0 à la 2.0 avec l'ajout du champ optionnel supplier_id.

```
+ supplier_id: Optional[str]
```

Stratégie de Migration

Rétrocompatibilité : Les flux V1 restent valides sans modification.

Validation Dynamique : Sélection du contrat ligne par ligne.

Zéro Big Bang : Coexistence possible des producteurs V1 et V2.

FLUX D'INGESTION (DAG 1 & 2)

1. Ingestion

Détection CSV, validation Pydantic et séparation immédiate Valid vs Rejected.

2. Persistance

UPSERT PostgreSQL pour garantir l'idempotence et export analytique Parquet.

3. Publication

Émission de Datasets Airflow déclenchant les analyses avales automatiquement.

Un design découplé pour une scalabilité horizontale du pipeline.

ANALYTICS INVENTAIRE (DAG 3)

Moteur DuckDB sur Parquet

Traitement des KPIs directement sur les fichiers de données sans serveur dédié.

- Calcul du stock par SKU
- Alerting sous-seuil (Stock < Min)
- Reporting CSV automatisé

9ms

TEMPS DE CALCUL (100K LIGNES)

REJEU DES REJETS (DAG 4)

Gérer l'Imprévu Asynchrone

Problème : Un mouvement logistique est reçu, mais son identifiant (SKU) n'existe pas encore au catalogue.

Ancien comportement : Ligne corrompue ou crash de la chaîne entière.

Solution DLQ : Isolation en temps réel des "unknown_sku" dans une table dédiée (Dead Letter Queue).

Résilience : Le DAG de rejeu (DAG 4) récupère automatiquement ces données dès l'intégration du catalogue.

Boucle de Rémédiation

Traitement de la "dette fonctionnelle" générée par l'asynchronisme :

- Scan des rejets PENDING
- Vérification de l'arrivée du SKU
- Re-insertion automatique dans les mouvements
- Statut REPLAYED pour auditabilité

"Aucune donnée n'est perdue, elle est simplement mise en attente de résolution."

Stockage Hybride : OLTP & OLAP

Opérationnel (PostgreSQL)

- Gestion de l'état en temps réel.
- Garanties ACID pour les transactions.
- Vérification de l'intégrité référentielle (Clés étrangères, Contraintes uniques).

Analytique (DuckDB + Parquet)

- Format en colonnes hautement compressé.
- Lectures agrégées massives sans pénaliser la production.
- Requêtes vectorisées foudroyantes pour le Reporting.

"Le compromis idéal : on n'interroge plus la base de production pour faire des bilans mensuels."

LogiStore en Chiffres

100k

Mouvements Validés

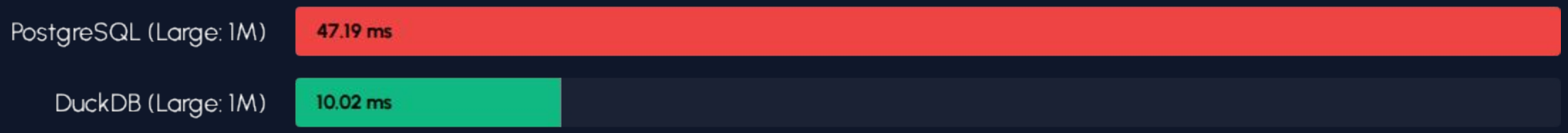
Preuve par l'exécution

- **Catalogue :** 500 produits ingérés.
- **Mouvements directs :** 91 985 lignes insérées sans erreur.
- **Quarantaine :** 40 075 mouvements isolés (unknown_sku).
- **Après Rejeu :** 40 075 lignes réintégrées avec statut REPLAYED.

"100% des données ont finalement été intégrées et réconciliées, de manière automatisée."

BENCHMARK SQL VS DUCKDB

Comparaison des performances sur la requête analytique temporelle (en ms) :



Small
DuckDB 5.0x plus rapide

Medium
DuckDB 4.7x plus rapide

Large
DuckDB 4.7x plus rapide

FIABILITÉ & QUALITÉ

Assurance Qualité

- **Tests Unitaires :** Validation contrats V1/V2 et règles métier.
- **CI/CD :** GitHub Actions automatisant les tests et le déploiement.
- **Idempotence :** Capacité à rejouer n'importe quel DAG sans doublons.

Data Quality by Design

Plan d'Industrialisation

Q1 - Cloud Native

Migration de la stack sur environnement Cloud (AWS S3 pour Parquet, RDS pour PostgreSQL).

Q2 - Transformation

Remplacement des requêtes analytiques Python brutes par l'orchestration via **dbt**.

Q3 - Streaming

Transition vers Apache Kafka pour gérer les mouvements de stock en temps réel vs Batch.

Q4 - Monitoring

Mise en place d'alerting Grafana branché directement sur les volumes de la zone de quarantaine (DLQ).

Bilan et Questions

La donnée n'est utile que si elle est digne de confiance.

LogiStore restaure cette confiance.

- **Business :** Fiabilité des stocks, fin des ruptures non anticipées.
- **Ops :** Dépannage simplifié (logs clairs, lignage Airflow).
- **Tech :** Architecture découplée, asynchrone et scalable.

Documentation
https://github.com/zarroukmh/Logistore/